FACE EMOTION DETECTION

GURU KANISHKAR R (241001068)

ABSTRACT

MoodMirror is a client-side, browser-based web application that detects human faces in a photograph and recognises the facial expression shown on each face. It is a typical web-programming project: the interface is written with HTML5, CSS3 and modern JavaScript, organised as a React single-page application, while the intelligence comes from pre-trained neural networks that are downloaded and executed inside the browser. Judging expressions by eye is subjective and cannot be repeated or measured. MoodMirror addresses this by analysing a captured or uploaded photo and reporting a confidence score for each of seven basic expressions, so that the result is objective, quick to obtain and easy to understand.

The application follows a step-wise request–response flow that is driven entirely by user events in the page. The user captures a photo with the camera or uploads a JPG, PNG or WEBP image through a file input, and the image is shown in the Workspace component. The AI models are loaded lazily, only when they are first needed, using asynchronous HTTP requests and a progress indicator. A Single Shot MultiBox Detector with a MobileNet V1 backbone (SSD MobileNet V1) then locates the faces, a 68-point landmark network marks facial key-points, and a convolutional FaceExpression network predicts the probability of happy, sad, angry, neutral, fearful, disgusted and surprised. The scores are converted to percentages, ranked, and rendered by React as a primary expression card, a face bounding box on the photo and an expression breakdown with one bar per emotion.

MoodMirror is built with React and the @vladmandic/face-api library, which runs on TensorFlow.js with WebGL acceleration. Because all inference takes place inside the user's browser, there is no back-end server, no database and no API call that carries the photo, so images are never uploaded, which protects privacy, and an analysis takes about one second. The interface uses state-driven rendering, CSS-based dark and light themes and in-page navigation to Home, How It Works and About sections. The application is useful for learning web programming together with computer vision, for interactive demonstrations of emotion recognition and as a base for emotion-aware web interfaces.

1. INTRODUCTION

Human emotion is communicated largely through the face. Smiles, frowns, raised eyebrows and tense lips carry information that is useful in education, customer feedback, human–computer interaction, healthcare support and entertainment. Doing this by eye is subjective, and doing it automatically requires a system that can first find a face in an image and then interpret its expression. Facial Expression Recognition (FER) is a classic computer vision problem that combines face detection, feature extraction and classification.

From a web programming point of view, the challenge is to deliver such a capability as an ordinary web page. Traditionally the image would be posted to a server, processed by a back-end program and returned as a response. MoodMirror instead uses the client-side model

of the modern web: the page is served as static HTML, CSS and JavaScript files, and everything else happens in the browser. The user opens the application, captures a live photo or uploads an existing image, and presses “Analyze”. The click event triggers an asynchronous JavaScript function that detects up to two faces, and React re-renders the page to draw a labelled bounding box around each face and to show the dominant expression with its confidence together with the score of all seven expressions. No installation is needed, no server is involved and the photo stays on the user’s device. The project therefore demonstrates how a modern deep-learning vision pipeline can be delivered as a lightweight, private and easy-to-use web application using standard web technologies.

2. PROJECT ARCHITECTURE

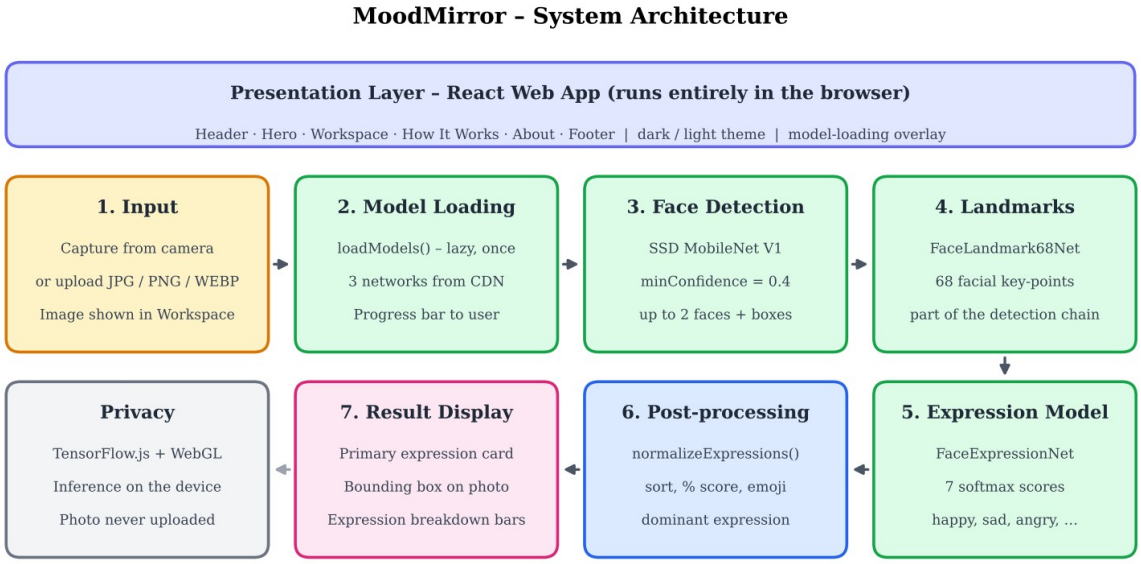


Fig. 1: System architecture of MoodMirror

MoodMirror follows a layered, component-based architecture. The presentation layer is a React single-page application (SPA) made of reusable components: Header, Hero, Workspace, How It Works, About and Footer. The theme (dark or light) and the model-loading state are kept as React state in the root App component and are passed down to child components as props, while callback functions are passed down so that child components can report user actions back to the parent. This one-way data flow keeps the interface consistent with the underlying state.

When the user starts an analysis, the service layer, a plain JavaScript module, downloads the face detector, landmark network and expression network over HTTP from a CDN once and caches the loaded state in module-level variables, so repeated clicks never download the models twice. The input image element is passed to the SSD MobileNet V1 detector, which returns face boxes above a confidence of 0.4; the landmark and expression networks then run on each detected face. A post-processing function converts the seven raw probabilities into a ranked list with label, emoji, colour and percentage, and selects the dominant expression. Finally, the results are stored in state and React renders them as the

primary-expression card, the face box on the photo and the expression bars. The whole pipeline executes on the client through TensorFlow.js and WebGL.

3. USER INTERFACE SCREENSHOTS

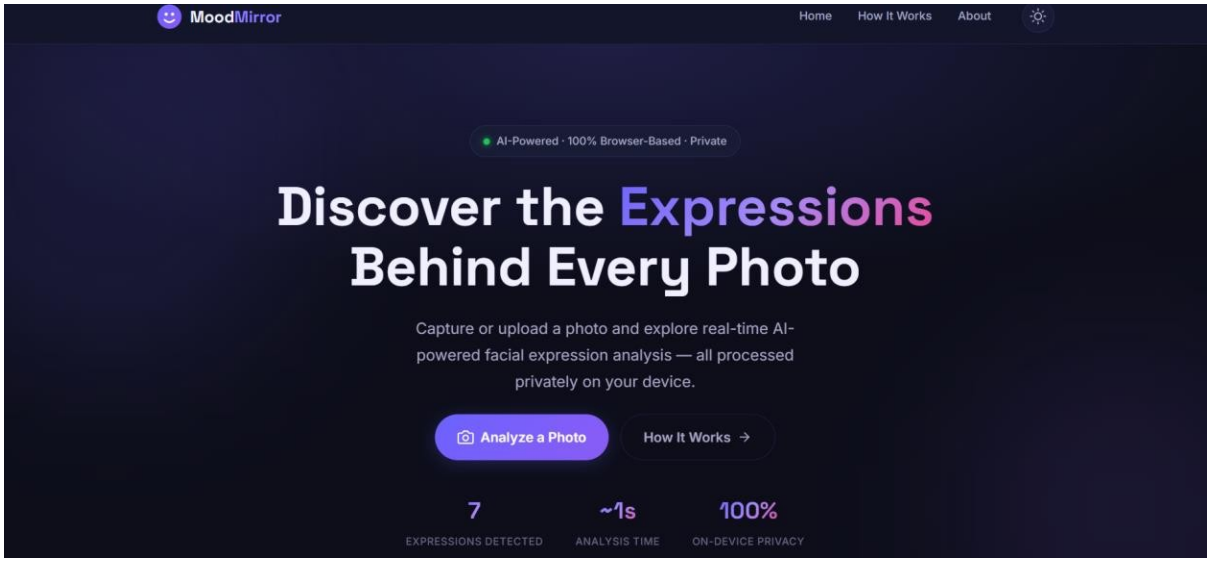


Fig. 2: Home page of MoodMirror (hero section)

The home page is the landing section of the single-page application. It is built from semantic HTML elements styled with CSS and carries the headline “Discover the Expressions Behind Every Photo” and the badge “AI-Powered · 100% Browser-Based · Private”. The “Analyze a Photo” button has a click handler that starts the analysis workflow and loads the models, while “How It Works” is an in-page anchor link that scrolls smoothly to the explanation section. Three statistics summarise the system: 7 expressions detected, about 1 second analysis time and 100% on-device privacy. The navigation bar provides Home, How It Works and About links and a button that toggles between dark and light themes.



Fig. 3: Analysis results for a sample photo

After analysis, the results panel is produced by conditional rendering: it appears only when the application state holds a result. The photo is shown with a colour-coded bounding box labelled “Face 1”, positioned over the image with absolute CSS positioning using the box coordinates returned by the detector. The Primary Expression card shows the dominant expression, here Happy with 100% confidence, together with a circular confidence meter. The face detection confidence is reported separately as 99%. The Expression Breakdown is generated by mapping over the ranked array, so each of the seven expressions gets an emoji, a progress bar whose width is bound to its percentage, and a percentage label; the top expression is highlighted with a “TOP” tag, and the others (Neutral, Disgusted, Sad, Angry and so on) are close to 0% for this clearly smiling face.

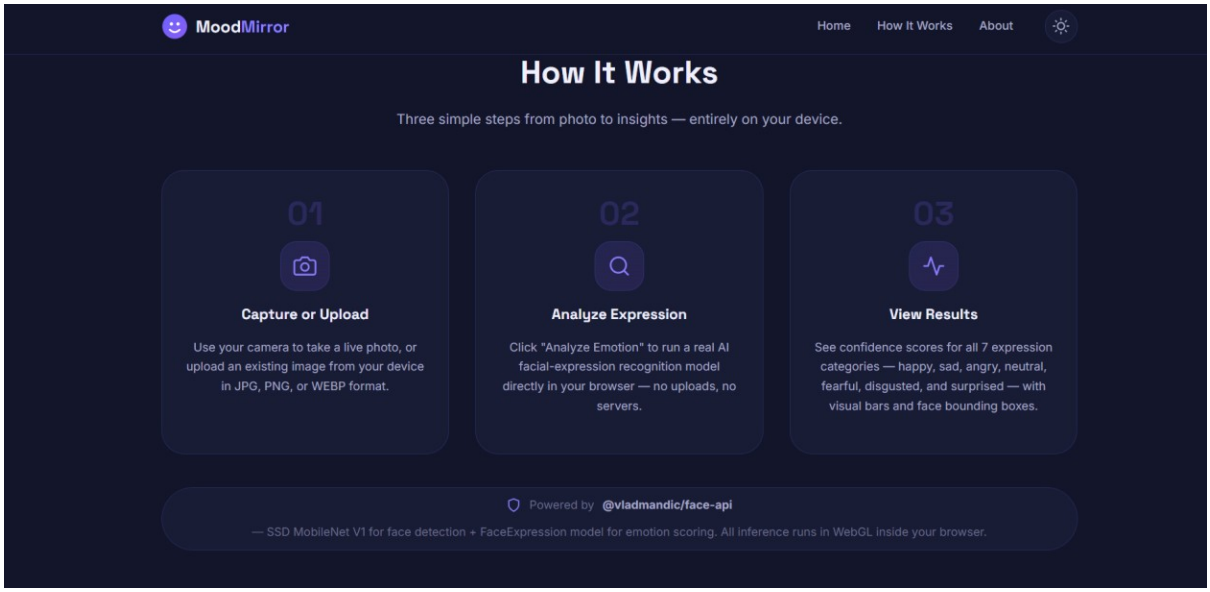


Fig. 4: How It Works section

The How It Works section is a static, responsive card layout that explains the process in three steps: Capture or Upload a photo (JPG, PNG or WEBP) through the camera or a file input, Analyze Expression using a real AI facial-expression recognition model that runs directly in the browser with no uploads and no servers, and View Results with confidence scores for all seven expression categories, visual bars and face bounding boxes. A footer note states that the application is powered by @vladmandic/face-api, with SSD MobileNet V1 for face detection and the FaceExpression model for emotion scoring, and that all inference runs in WebGL inside the browser.

4. TECHNIQUES USED

The table below lists the web programming and computer vision techniques used in MoodMirror, the stage and function where each is used, and its purpose.

No.	Technique	Where it is used (stage / function)	Purpose
1	Image acquisition and decoding	Input – browser camera / file upload; analyzeImage(imageElement)	Uses the browser's camera and file-input APIs and the DOM image element so that a live capture or a JPG / PNG / WEBP file can be decoded and processed on the device
2	Face detection with SSD MobileNet V1	Detection – faceapi.detectAllFaces, SsdMobilenetv1Options	An asynchronous call to a client-side JavaScript library; a single-shot convolutional detector finds every face in the photo and returns a bounding box and detection score
3	Confidence thresholding	Detection – minConfidence = 0.4, maxResults = 2	A configuration object passed to the detector that discards weak detections and limits the number of faces rendered, so only reliable faces reach the interface
4	Bounding-box localisation	Detection / display – det.detection.box, resizeResults	Returns the face position (x, y, width, height), rescaled to the displayed image size and drawn as the "Face 1" overlay using CSS positioning
5	68-point facial landmark detection	Landmarks – faceLandmark68Net, withFaceLandmarks()	Chained onto the detection call (method chaining on a Promise-based API) to locate 68 key-points on each face: eyes, eyebrows, nose, mouth and jaw
6	Facial expression recognition (CNN classifier)	Classification – faceExpressionNet, withFaceExpressions()	Predicts a probability for each of the 7 expressions: happy, sad, angry, neutral, fearful, disgusted, surprised, and returns them as a JavaScript object
7	Softmax probability normalisation and ranking	Post-processing – normalizeExpressions()	Array methods (filter, map, sort) turn the 7 scores into percentages, attach label, emoji and colour, and sort them so React can render the ranked list and find the dominant expression
8	WebGL-accelerated on-device inference	Runtime – TensorFlow.js backend used by face-api	Runs all neural networks on the browser GPU through the WebGL API, so the photo never leaves the device and analysis takes about a second
9	Lazy, idempotent model loading	Start-up – loadModels(onProgress), ensureModelsLoaded()	Async / await with a cached Promise downloads the three model files once, only when needed, updates React state for a progress bar and allows a retry if loading fails

CONCLUSION:

This project presented MoodMirror, a browser-based face emotion detection application built with React and the @vladmandic/face-api library. As a web programming exercise it combines component-based design, state and props, event handling, conditional rendering, asynchronous JavaScript (Promises and async / await), lazy loading and CSS theming. The system detects faces with SSD MobileNet V1 using a confidence threshold of 0.4, finds 68 facial landmarks and classifies the expression into seven categories (happy, sad,

angry, neutral, fearful, disgusted and surprised) with a convolutional expression network. The results are shown as a primary expression with its confidence, a face bounding box and a ranked expression breakdown. Because the models run on TensorFlow.js with WebGL, all processing is done on the user's device in about one second and photos are never uploaded, which keeps the tool private and easy to deploy as a set of static files with no server. In testing with a clear frontal photograph the system correctly reported Happy with a face detection confidence of 99%. Overall, MoodMirror meets its objective of making facial expression analysis simple, fast and private, and shows how web technologies and deep learning can be combined into a practical web application.

Applications

- Learning tool for web programming and computer vision, showing how a React interface can host face detection and expression recognition in a web page.
- Interactive demonstrations and classroom activities on emotion recognition and client-side web development.
- Basis for emotion-aware web interfaces, such as feedback kiosks and adaptive learning tools.
- Privacy-friendly analysis of photos, since nothing is sent to a server.

Limitations

- Works best on well-lit, front-facing faces; head tilt, occlusion by hands or masks and poor lighting reduce accuracy.
- Only seven basic expressions are recognised, so subtle or mixed expressions may be classified wrongly.
- The output describes the visible facial expression, not a person's true feelings.
- Analysis is done on a single photo, and at most two faces are processed at a time.
- The models must be downloaded over HTTP once, so an internet connection is needed on first use; performance also depends on the browser's WebGL support and the device GPU.

Future Enhancements

- Real-time emotion detection on live camera video using a continuous camera stream.
- Support for more faces and a history of past results stored in the browser (for example with localStorage or IndexedDB).
- Models trained on more diverse data to improve accuracy across ages, skin tones and lighting.
- Exporting results as a downloadable report or image, and making the application installable as a Progressive Web App that works offline.

ANNEXURE: CODE

This annexure gives the essential source files of MoodMirror, grouped by module, followed by the most important parts of the code. The code is shortened for readability: comments are added and optional features are left out.

A. Essential project files

No.	File name	Purpose
	Entry point	
1	main.jsx	Entry script that mounts the React application into the page's root DOM element inside React.StrictMode and loads the global styles.
2	App.jsx	Root component: holds the theme (dark / light) and lazy model-loading state and defines the page layout.
	AI service (src/services)	
3	emotionDetection.js	JavaScript module that loads the face-api models, detects faces, scores 7 expressions and formats the result.
	Components (src/components)	
4	Header.jsx	Top navigation (Home, How It Works, About) and the dark / light theme toggle.
5	Hero.jsx	Landing section with the headline, "Analyze a Photo" button and key statistics.
6	Workspace.jsx	Photo capture / upload area, "Analyze" action and the analysis results panel.
7	ModelLoader.jsx	Overlay with a progress bar shown while the AI models are downloading.
8	HowItWorks.jsx	Three-step explanation: capture or upload, analyze, view results.
9	About.jsx / Footer.jsx	Project information and page footer.
	Styling	
10	styles.css	Dark and light themes, responsive layout, cards, buttons and expression bars.

B. Important project code

The code below is taken from emotionDetection.js, App.jsx and main.jsx.

B.1 Loading the face-api models (emotionDetection.js – loadModels)

```
export async function loadModels(onProgress) {
  if (modelsLoaded) return;                 // already loaded
  if (loadingPromise) return loadingPromise; // load in progress
  loadingPromise = (async () => {
    try {
      const report = (p, m) => onProgress && onProgress(p, m);
      report(10, 'Loading face detector…');
      await faceapi.nets.ssdMobilenetv1.loadFromUri(MODEL_URL);
      report(55, 'Loading landmark model…');
      await faceapi.nets.faceLandmark68Net.loadFromUri(MODEL_URL);
      report(80, 'Loading expression model…');
      await faceapi.nets.faceExpressionNet.loadFromUri(MODEL_URL);
      report(100, 'Models ready!');
      modelsLoaded = true;
    } catch (err) {
      loadingPromise = null;                // allow retry
```

```
      throw new ModelLoadError(err.message);
    }
  })();
  return loadingPromise;
}
```

B.2 Face detection and expression analysis (emotionDetection.js – analyzeImage)

```
export async function analyzeImage(imageElement) {
  // SSD MobileNet V1 face detector settings
  const options = new faceapi.SsdMobilenetv1Options({
    minConfidence: 0.4, maxResults: 2,
  });
  // detect faces -> 68 landmarks -> 7 expression scores
  const detections = await faceapi
    .detectAllFaces(imageElement, options)
    .withFaceLandmarks()
    .withFaceExpressions();
  if (!detections || detections.length === 0)
    return { faces: [], noFaceDetected: true };
  const displaySize = {
    width: imageElement.naturalWidth || imageElement.width,
    height: imageElement.naturalHeight || imageElement.height,
  };
  const resized = faceapi.resizeResults(detections, displaySize);
  const faces = resized.map((det, idx) => {
    const expressions = normalizeExpressions(det.expressions);
    const dominant = expressions.reduce(
      (a, b) => (a.score > b.score ? a : b));
    return { id: idx + 1, box: det.detection.box, expressions,
             dominant, confidence: det.detection.score };
  });
  return { faces, noFaceDetected: false };
}
```

B.3 Expression metadata and ranking (emotionDetection.js – normalizeExpressions)

```
const EMOTION_META = {
  happy:     { label: 'Happy',     emoji: '😊', color: '#FBBF24' },
  sad:       { label: 'Sad',       emoji: '😊', color: '#60A5FA' },
  angry:     { label: 'Angry',     emoji: '😊', color: '#F87171' },
  neutral:   { label: 'Neutral',   emoji: '😊', color: '#94A3B8' },
  fearful:   { label: 'Fearful',   emoji: '😊', color: '#A78BFA' },
  disgusted: { label: 'Disgusted', emoji: '😊', color: '#34D399' },
  surprised: { label: 'Surprised', emoji: '😊', color: '#FB923C' },
};

function normalizeExpressions(expObj) {
  return Object.entries(expObj)
    .filter(([key]) => key in EMOTION_META)
    .map(([key, raw]) => ({
      key, ...EMOTION_META[key],
      score: raw, percent: Math.round(raw * 100),
    }))
    .sort((a, b) => b.score - a.score);   // highest first
}
```

B.4 Lazy model loading and page layout (App.jsx)

```
const ensureModelsLoaded = useCallback(async () => {
  if (areModelsLoaded()) return true;
  if (modelState.status === 'loading') return false;
  setModelState({ status: 'loading', progress: 5,
    message: 'Starting AI engine…', error: null });
  try {
    await loadModels((pct, msg) =>
      setModelState(p => ({ ...p, progress: pct, message: msg })));
```

```
      setModelState({ status: 'ready', progress: 100,
        message: 'Models ready!', error: null });
      return true;
    } catch (err) {
      setModelState({ status: 'error', progress: 0, message: '',
        error: err.message || 'Failed to load AI models.' });
      return false;
    }
  }, [modelState.status]);

  return (
    <div className="app-root">
      <Header theme={theme} onToggleTheme={toggleTheme} />
      {modelState.status === 'loading' && (
        <ModelLoader progress={modelState.progress}
          message={modelState.message} />)}
      <main>
        <Hero onAnalyzeClick={ensureModelsLoaded} />
        <Workspace modelState={modelState}
          onEnsureModels={ensureModelsLoaded} />
        <HowItWorks />
        <About />
      </main>
      <Footer />
    </div>
  );
```

B.5 Application entry point (main.jsx)

```
import React from 'react';
import ReactDOM from 'react-dom/client';
import App from './App.jsx';
import './styles.css';

ReactDOM.createRoot(document.getElementById('root')).render(
  <React.StrictMode>
    <App />
  </React.StrictMode>
);
```